Learning When to Verify Under a Budget: Process Rewards, Representations, and Length Generalization

Raj Kashikar

Abstract

Reasoning systems must decide not only how to solve a problem but also when to spend effort checking their own work. We study this decision in a small testbed where everything of interest is exactly measurable. An agent executes an arithmetic chain that may contain one silent fault, sees a noisy one-bit cue per step, and divides a fixed budget between executing, verifying and repairing steps. We train small Double DQN agents with outcome-only, naive process and monitorability-aware rewards and several observation encoders, and evaluate them on chains up to 2.3× longer than in training, with 10 seeds and paired statistics. We report four findings.

1. Process rewards make checking learnable and keep it from collapsing. With length-invariant observations, outcome-only agents find checking policies but have abandoned them in 60% of late checkpoints, against 0% under the monitorability-aware reward. Ablations attribute nearly all of that reward's benefit to one term that pays for checking cued steps.
2. The reward is partly gamed. It pays for *catching* a fault, and agents repeatedly drift into policies that detect faults without repairing them. Paying for the repair instead removes this collapse (21% → 1% of late checkpoints) and makes the last checkpoint far more reliable.
3. Length generalization is decided mainly by representation. With absolute position features every held-out observation is novel, and every reward scheme falls *below* a never-check baseline on lengths 10–14. With length-invariant features the same reward keeps 0.86 success at lengths 7–9, within a few points of a hand-coded checker, and 0.74 at 10–14. A feature that adds information but goes out of range on long chains made generalization worse, as an observation-coverage analysis predicted before the experiment.
4. DQN policies churn throughout training, so last-checkpoint evaluation understates what runs learn.

These results revise an earlier version of this study, which attributed the length-generalization failure to the reward.

1 Introduction

Reinforcement learning has produced language models that spend test-time compute on long chains of reasoning, including checking and revising their own steps [OpenAI, 2024, DeepSeek-AI, 2025, Snell et al., 2024]. Two lines of work make this behaviour a target in its own right. Process supervision rewards individual reasoning steps rather than only final answers [Uesato et al., 2022, Lightman et al., 2024]. Work on chain-of-thought monitorability

Preprint.

asks whether reasoning traces stay legible enough to oversee, and warns that optimising against a monitor can make them less so [Baker et al., 2025, Korbak et al., 2025, OpenAI, 2025]. Both lines touch a basic metareasoning question [Russell and Wefald, 1991, Hay et al., 2012]: with a finite budget, *which* steps are worth checking, and does a learned answer to that question survive when the problem gets longer?

That question is hard to study in language models, where the location of an error, the value of a check and the faithfulness of a trace [Turpin et al., 2023, Lanham et al., 2023, Chen et al., 2025] are all hard to measure. We therefore isolate it in a small testbed (§3). An agent executes an arithmetic chain that may contain one silent fault. It sees a noisy one-bit cue per step and can execute, verify, undo or commit, each non-commit action consuming a fixed budget. The task admits strong hand-coded policies (a cue-triggered checker solves 85–88% of episodes at every length we test), so failures of learning cannot be blamed on an impossible task. Because every quantity is exact, we can separate three things that are entangled in realistic settings: whether an agent learns to check at all, whether it checks the *right* steps, and whether the behaviour transfers to longer chains than it was trained on.

Contributions.

- A small, fast and fully tested environment for budgeted verification, with exact metrics for faults, detection, repair and cue-conditional checking, three reward schemes, several observation encoders and hand-coded reference policies (§3–4).
- Evidence that process rewards help agents *acquire and retain* checking. With absolute features, outcome-only agents never acquire it. With invariant features they find it but usually lose it again, while a monitorability-aware reward keeps agents checking (§6.1). Ablations trace almost all of that reward's benefit to a single term that pays for checking cued steps (§6.7).
- A reward side effect: because the monitorability-aware reward pays for catching a fault rather than fixing it, its agents repeatedly drift into policies that detect faults without repairing them (§6.2).
- Evidence that length generalization is dominated by representation. Absolute position features make every held-out observation novel and drive every reward scheme below a never-check baseline on long chains. A length-invariant encoder recovers most of the performance; the remaining gap comes from how agents spend their budget slack (§6.3–6.5).
- Follow-up interventions that test each explanation directly, robustness to shifts in fault rate, verification cost, budget and cue quality, and a training-stability analysis showing pervasive policy churn [Schaul et al., 2022] (§6.6–6.9).

A correction. An earlier version of this project, using the same environment and rewards, concluded that monitorability-aware RL "remains brittle under longer held-out chains". A controlled re-run reversed that conclusion. The main cause was the absolute position features in the observation, compounded by 4× less training and last-checkpoint evaluation. The earlier pipeline also used a localization metric that gave credit for faults nobody checked, and it reused training seeds for one seed's evaluation episodes (harmless here, but a protocol error). Appendix E lists every change. We report this because, at larger scale, the same confounds (positional shortcuts and unstable policies) can make a reward look responsible for failures it did not cause.

2 Related Work

Process supervision and verification. Step-level feedback improves the reliability of mathematical reasoning [Uesato et al., 2022, Lightman et al., 2024], and RL on verifiable tasks can elicit self-checking [DeepSeek-AI, 2025, Xie et al., 2025]. These works ask whether process feedback helps. We ask a narrower question in a controlled setting: when a process reward helps an agent learn to check, what has it learned, and does that transfer? Our learned rewards are hand-specified with privileged access to step correctness. They therefore

stand in for an *accurate* process reward model and measure what such a model could do at best.

Monitorability and faithfulness. Reasoning traces can be informative about model behaviour [Baker et al., 2025, Korbak et al., 2025], but they are not guaranteed to be faithful [Turpin et al., 2023, Lanham et al., 2023, Chen et al., 2025], and optimisation pressure on traces can induce obfuscation [Baker et al., 2025]. Our "monitorability-aware" reward is a toy instance of rewarding traces that remain locally diagnosable: it pays for checks that catch or rule out faults and penalises long unchecked stretches. Our environment has no notion of a model hiding its reasoning, so we make no claims about faithfulness.

Metareasoning and adaptive computation. Deciding which computation to perform next is classical metareasoning [Russell and Wefald, 1991, Hay et al., 2012], and adaptive computation time learns how long to think [Graves, 2016]. Our agents learn a value-of-information policy for checking, without being given the posterior over fault locations.

Generalization and evaluation in RL. Deep RL agents overfit their training distribution [Cobbe et al., 2019, Kirk et al., 2023], and neural networks often fail to generalize to longer inputs than seen in training [Anil et al., 2022, Delétang et al., 2023]. Our length-generalization result is a small, controlled instance of this: it comes down to whether the observation lets the policy depend on absolute position. Methodologically we follow recommendations to report many seeds with interval estimates rather than point estimates [Henderson et al., 2018, Agarwal et al., 2021]. We also document policy churn [Schaul et al., 2022]: greedy policies that change abruptly between updates, which makes last-checkpoint evaluation unreliable.

3 A Testbed for Learning When to Verify

Task. Each episode is an arithmetic chain $s_0 \to s_1 \to \cdots \to s_d$ of depth d: the start value $s_0 \in \{2, \ldots, 6\}$ is followed by d randomly sampled transitions (add, subtract, or multiply by a small constant). With probability $p_f = 0.8$, exactly one transition h is *faulty*: until the fault has been located, executing it applies a corrupted operation, so the agent's running value silently diverges from the true value. Conditional on a fault, h is drawn from a prior that peaks in the middle of the chain. Each transition carries a one-bit *cue* c_t that fires with probability 0.85 on the faulty transition and with probability 0.15 on every other transition. The cue plays the role of a cheap, noisy signal that a reasoning step looks suspicious.

Actions and budget. The agent chooses among four actions. `THINK` executes the next transition. `VERIFY` exactly checks the most recently executed transition and returns SAFE or UNSAFE; the check is *local*, so it does not detect corruption inherited from an earlier unchecked fault. The first UNSAFE result locates the fault, after which re-executing it (`REVISE`, which undoes the last step, followed by `THINK`) applies the correct operation. `COMMIT` ends the episode, which succeeds if and only if all d transitions have been executed and the final value is correct. `THINK`, `VERIFY` and `REVISE` each cost one unit of a budget of $d + 6$ units. The agent can therefore afford six actions beyond the chain itself: enough to check some steps and repair one fault, but not to check every step of a long chain.

The environment is deliberately small. All quantities of interest are exact: whether a fault was present, whether and where it was caught, and which steps were checked. The question is not whether the task is hard, but whether reinforcement learning (RL) finds and keeps the right *allocation* of checking effort, and whether that allocation transfers to longer chains.

Observations. The agent never observes the running value, the true value or the fault location. We compare two observation encoders (Table 1). Both expose the local information a hand-coded checking policy needs (cue, budget slack, unchecked gap, last check result). The invariant encoder is a strict reduction of the absolute one: it replaces position and depth by two boundary flags and drops the budget fraction, so the number of remaining steps cannot be recovered from it.

Table 1: Observation features. The invariant encoder replaces absolute position and length by boundary flags, so remaining length cannot be recovered from it.

Information	Absolute encoder	Invariant encoder
Position	$t/9$, depth $d/9$	$[t{=}0]$, $[t{=}d]$
Budget	b_t/B, $\mathrm{clip}(0.5 + \mathrm{slack}/2B, 0, 1)$	$\mathrm{clip}(\mathrm{slack}, \pm 10)/10$
Unchecked gap	$g_t/9$	$\min(g_t, 3)/3$
Cue, revised flag, last check	shared	shared

- **absolute** (10-d): normalised position $t/9$, depth $d/9$, budget fraction, normalised budget slack, normalised unchecked gap, cue, a revised-last-step flag and a one-hot of the last verification result. This is the encoding used in the original version of this study.
- **invariant** (9-d): flags for $t = 0$ and $t = d$, budget slack $\mathrm{clip}(b_t - (d - t), -10, 10)/10$, unchecked gap $\min(g_t, 3)/3$, cue, the revised flag and the verification one-hot.

Here b_t is the remaining budget and $g_t = t - \max(\text{checked indices})$ the number of steps executed since the highest-index checked step. Budget slack $b_t - (d - t)$ is the number of non-`THINK` actions still affordable. On held-out lengths the absolute encoding produces feature values never seen in training (for example $d/9 > 6/9$); the invariant encoding rarely does (§6.3). Neither encoding gives the agent memory of earlier UNSAFE results. As a consequence, a genuine repair (UNSAFE, `REVISE`, `THINK`) and a no-op `REVISE`+`THINK` after a SAFE check lead to identical observations (§6.9 adds a one-bit memory).

4 Rewards, Learner and Reference Policies

Base reward. All agents receive $+1$ for a successful commit, -1.2 for a failed one, and -0.04 per unit of budget spent.

Process rewards. We compare three reward schemes, keeping the coefficients of the original study:

- **Outcome-only**: the base reward alone.
- **Naive process**: $+0.015$ for each `THINK` that leaves the running value correct, and $+0.02$ for each first-time informative check. These are the kind of dense "step is correct" signals a process reward model provides [Uesato et al., 2022, Lightman et al., 2024].
- **Monitorability-aware**: rewards checks that make the trajectory more diagnosable:

$$r_t^{\mathrm{MA}} = 0.02\,\mathbb{1}[\text{safe check}] + 0.25\,\mathbb{1}[\text{fault caught}] + 0.18\,\mathbb{1}[\text{informative check}, c_t{=}1] - 0.02\,\mathbb{1}[\text{safe check}, c_t{=}0] - 0.06\,\mathbb{1}[\text{wasted check}] - 0.3\,\frac{g_{t+1} - g_t}{d}. \tag{1}$$

  A check is *informative* if it is the first check of that step and either confirms a correct value or catches the fault. It is *wasted* if it repeats a check, or checks a step downstream of an undetected fault.

Two properties of this reward, inherited unchanged from the original study, matter for what follows. First, the fault bonus is paid when the fault is *caught*, whether or not it is then repaired. Second, the gap term telescopes to $-0.3\,g_T/d$ over an episode. It is therefore close to potential-based shaping [Ng et al., 1999] plus a terminal penalty that depends only on how far the last *checked index* is from the end. One check of the final step cancels it, and any action that lowers g is refunded, including a `REVISE` or a check that is otherwise "wasted". For a check downstream of a hidden fault, the refund often exceeds the 0.06 waste penalty. The non-gap terms are not potential-based and can change the optimal policy. Both process rewards use privileged information (the hidden correctness of the running value), standing in for an accurate process reward model.

Learner. Every learned agent is a masked Double DQN [Mnih et al., 2015, van Hasselt et al., 2016] with a 2×64 ReLU MLP (∼5k parameters), Huber loss, Adam (learning rate 10^{-3}), a 25k-transition uniform replay buffer, batch size 64, a hard target update every 200 environment steps, and ϵ-greedy exploration annealed linearly from 0.5 to 0.02 over the first 85% of training. Unavailable actions are masked both when acting and in the bootstrap target. Each run trains for 20,000 episodes on chains of length 3–6. These are the original study's hyperparameters, unchanged except for 4× more training, and they were not tuned for any condition. §6.8 checks the encoder comparison under other learning rates and target-update schemes.

Reference policies. We also evaluate four hand-coded policies that see only observable quantities. *Always THINK* never checks. *Verify every step* checks each new step. *Cue-triggered* checks a new step if and only if its cue fired. *Budget-aware* checks cued steps whenever a repair stays affordable, and also checks un-cued steps while the budget slack exceeds that requirement by a margin m. We pick m=1, the value that maximises success on training lengths (`scripts/tune_heuristic.py`). All four policies repair immediately after an UNSAFE check.

5 Evaluation Protocol

Seeds and episodes. Every learned condition is trained with 10 independent seeds. Each policy is evaluated on 500 episodes per seed and setting. Training, learning-curve validation and final evaluation draw episodes from independent random-number streams. Policies observe only depth, position, budget, cue and check results, so on training lengths the same small set of task configurations recurs in all three streams; training-length evaluation is therefore in-distribution by design, and held-out lengths are the only genuinely unseen configurations. Methods are evaluated on identical episodes for a given seed, so comparisons are paired.

Settings. We report success on training lengths (3–6), on held-out lengths 7–9, and on lengths 10–14. On 7–9 we additionally shift the fault rate ($p_f \in \{0.6, 0.9\}$), the verification cost ($c_V \in \{2, 3\}$), the budget slack (4 or 8) and the cue quality (clean: 0.95/0.05; noisy: 0.60/0.35), without retraining.

Checkpoint selection. DQN policies on this task change abruptly during training (§6.8), so the last checkpoint is a noisy sample of what a run has learned. We greedily evaluate every agent on 200 validation episodes of *training-length* chains every 1,000 episodes, and keep the checkpoint with the highest validation success. Held-out lengths are never used for selection; the config loader rejects any selection setting that is not the training distribution. Appendix C reports every number again for the last checkpoint: the encoder and reward conclusions do not change, but parity with the cue-triggered heuristic holds only for selected checkpoints (§6.8).

Metrics and statistics. Beyond success we report *fault detection*, P(an UNSAFE check occurs | fault); *repair completion*, P(success | fault detected); the number of *non-repair* `REVISE`s (those not directly following an UNSAFE check); and *cue-conditional checking*, the probability of choosing `VERIFY` at a decision point whose current step is unchecked, split by its cue c. The last is pooled over decision points, so a step revisited after a `REVISE` counts again. Intervals are 95% Student-t intervals of the mean over seeds; with 10 seeds a percentile bootstrap undercovers noticeably [Agarwal et al., 2021]. Paired differences between methods report the mean over seeds with a t-interval, the number of seeds favouring each method, and an exact two-sided sign-flip permutation p-value.

6 Results

Table 2 summarises the main experiment and Figure 1 shows the headline result. Unless stated otherwise, numbers are means over 10 seeds with 95% t-intervals, evaluated with the

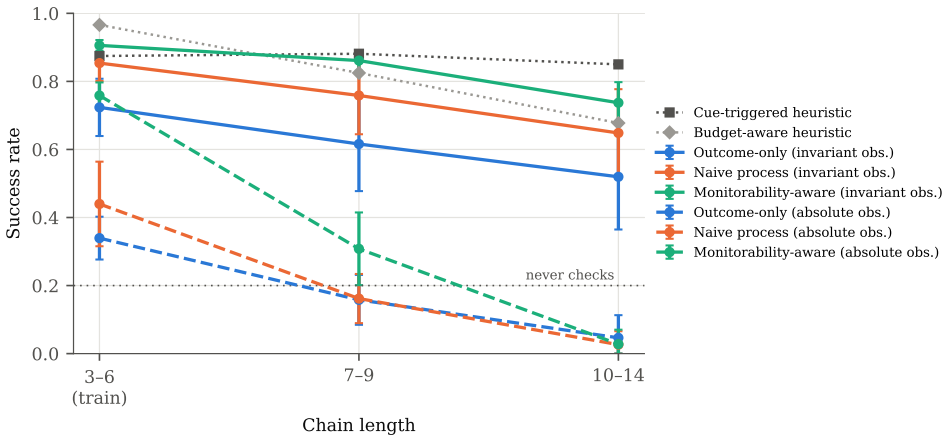


Figure 1: Success on training lengths (3–6) and on longer held-out chains. Colour encodes the reward and line style the observation encoder (solid: invariant, dashed: absolute); error bars are 95% t-intervals over 10 seeds. With absolute features every reward scheme drops below a policy that never checks. With invariant features the monitorability-aware agent comes within a few points of the cue-triggered heuristic at lengths 7–9.

Table 2: Main results (mean over 10 seeds, 95% t-interval). "Detect" is the fraction of faulty episodes in which a check caught the fault; "cost-adj." is success minus 0.05× budget spent. Top block: hand-coded references; bottom block: learned agents (checkpoint selected on training lengths only).

Method	Success 3–6	Success 7–9	Success 10–14	Detect 7–9	Cost-adj. 7–9
Random	0.00 [0.00, 0.01]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.02 [0.01, 0.02]	-0.09 [-0.09, -0.08]
Always THINK	0.20 [0.19, 0.21]	0.20 [0.19, 0.22]	0.20 [0.18, 0.21]	0.00 [0.00, 0.00]	-0.20 [-0.21, -0.18]
Verify every step	0.80 [0.79, 0.81]	0.06 [0.06, 0.07]	0.00 [0.00, 0.00]	0.93 [0.91, 0.94]	-0.63 [-0.64, -0.63]
Cue-triggered	0.87 [0.86, 0.89]	0.88 [0.87, 0.90]	0.85 [0.84, 0.86]	0.85 [0.84, 0.87]	0.33 [0.31, 0.34]
Budget-aware	0.97 [0.96, 0.97]	0.82 [0.81, 0.84]	0.68 [0.66, 0.70]	0.78 [0.76, 0.80]	0.19 [0.17, 0.20]
Outcome (inv)	0.72 [0.64, 0.81]	0.62 [0.48, 0.75]	0.52 [0.36, 0.67]	0.61 [0.43, 0.80]	-0.02 [-0.16, 0.12]
Naive process (inv)	0.85 [0.80, 0.91]	0.76 [0.64, 0.87]	0.65 [0.52, 0.78]	0.76 [0.62, 0.90]	0.09 [-0.02, 0.21]
Monitorability (inv)	0.91 [0.89, 0.92]	0.86 [0.83, 0.89]	0.74 [0.68, 0.80]	0.85 [0.83, 0.88]	0.19 [0.15, 0.22]
Outcome (abs)	0.34 [0.28, 0.40]	0.16 [0.08, 0.23]	0.05 [0.00, 0.11]	0.09 [0.03, 0.16]	-0.27 [-0.39, -0.15]
Naive process (abs)	0.44 [0.32, 0.56]	0.16 [0.09, 0.23]	0.03 [0.00, 0.07]	0.30 [0.08, 0.52]	-0.32 [-0.44, -0.21]
Monitorability (abs)	0.76 [0.72, 0.80]	0.31 [0.20, 0.41]	0.03 [0.00, 0.07]	0.60 [0.43, 0.76]	-0.24 [-0.33, -0.15]

checkpoint selected on training-length validation. Paired differences give the mean difference over seeds, its interval, the number of seeds favouring the first method, and a sign-flip p-value.

6.1 Process rewards make checking learnable and reduce collapse

Never checking succeeds exactly when no fault is present, which happens 20% of the time. Any gain above that requires the agent to discover a three-step pattern: check a suspicious step, see UNSAFE, then undo and redo.

Absolute observations. Outcome-only agents essentially never acquire this pattern. In 0/10 seeds does any validation checkpoint exceed 0.5 success, and the selected checkpoints reach 0.34 [0.28, 0.40] on training lengths. The monitorability-aware reward reaches 0.76 [0.72, 0.80] (paired +0.42 [0.38, 0.46], 10/10 seeds, $p = 0.002$).

Invariant observations. Outcome-only agents *do* find checking policies: some validation checkpoint exceeds 0.5 in 9/10 seeds. They rarely keep them. After the first 5,000 episodes, 60% of their greedy checkpoints have stopped checking altogether (success ≤ 0.35 and fault detection below 0.2; Figure 2). Under the monitorability-aware reward 0% of late checkpoints

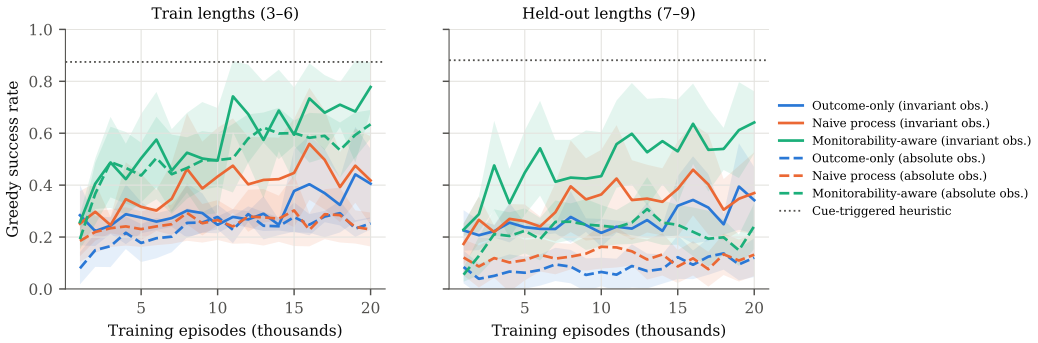


Figure 2: Greedy success during training (mean over 10 seeds, shaded 95% t-interval), evaluated every 1,000 episodes on validation episodes. The mean hides large per-seed swings (Figure 4).

have. At selected checkpoints, the monitorability-aware reward beats outcome-only by +0.18 [0.10, 0.26] ($p = 0.002$). Its advantage over naive process reward is smaller and less certain: +0.05 [-0.00, 0.11], 7/10 seeds, $p = 0.066$. Its training-length success of 0.91 [0.89, 0.92] lies between the cue-triggered heuristic (0.87) and the tuned budget-aware heuristic (0.97).

6.2 A side effect of the reward: detecting without repairing

The monitorability-aware agents' low-success checkpoints are not never-check policies. Of the late checkpoints of the invariant agent, 21% have success ≤ 0.35. Of these, 21% of all late checkpoints still catch at least half of all faults, but fail to complete the repair. This is what the reward pays for. The 0.25 fault bonus, plus 0.18 if the step was cued, is earned when the fault is caught, whether or not the agent then undoes and redoes the step. The effect is visible in the evaluated agents too (Appendix Table 9). Even on training lengths, where no input is out of range, the last checkpoints complete the repair after a caught fault in only 66% (absolute features) and 83% (invariant features) of episodes, against 100% for the hand-coded policies. Moving the fault bonus from detection to repair is a direct test of this explanation (§6.9).

6.3 Length generalization is mainly a representation problem

Figure 1 separates the effect of the reward from the effect of the observation encoding. With absolute features, success falls sharply on longer chains for every reward. On lengths 10–14 all three absolute-feature agents are *worse than never checking*: 0.05, 0.03 and 0.03, versus 0.20.

The explanation is that every observation the absolute encoder produces on held-out lengths is new: across rollouts of the hand-coded policies, 100% of decision points at lengths 7–9 have an observation vector that never occurs at lengths 3–6. For the invariant encoder the figure is 2% at 7–9 and 5% at 10–14. The absolute-feature policies act erratically on these inputs. For example, on lengths 10–14, 70% of monitorability-aware episodes end with a `COMMIT` before the chain is finished although the remaining budget could still finish it.

Changing only the observation encoder removes most of the failure. The monitorability-aware agent with invariant features keeps 0.86 [0.83, 0.89] at lengths 7–9 and 0.74 [0.68, 0.80] at 10–14. These are paired improvements over the absolute encoder of +0.55 [0.45, 0.66] and +0.71 [0.62, 0.80], in 10/10 seeds ($p = 0.002$). At lengths 7–9 the agent comes within -0.02 [-0.05, 0.01] of the cue-triggered heuristic. This is not a significant difference ($p = 0.166$), but it is not a demonstration of equivalence either.

The reward still matters under the invariant encoder. The monitorability-aware reward beats outcome-only at every length (+0.25, $p = 0.004$ at 7–9; +0.22, $p = 0.043$ at 10–14). Naive process reward falls between the two in point estimate, but with 10 seeds its separation

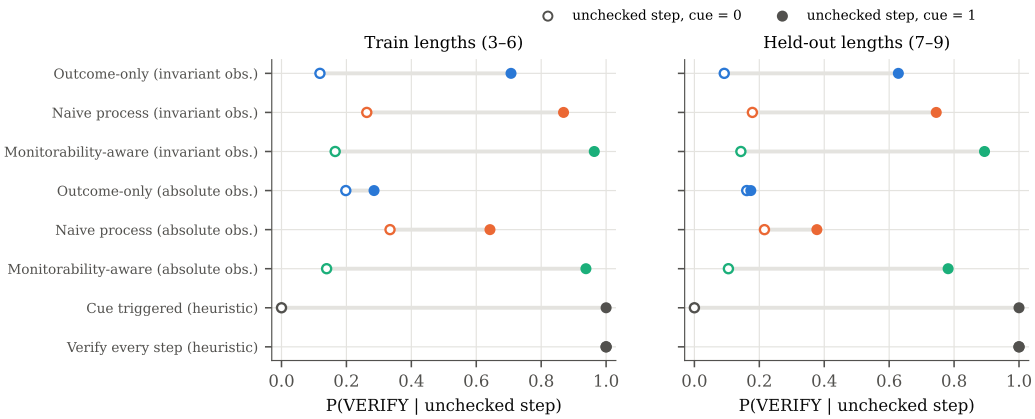


Figure 3: Cue-conditional checking: probability of choosing `VERIFY` at a decision point whose current step is unchecked, when that step's cue is off (open) or on (filled). The cue-triggered heuristic is perfectly cue-conditional by construction.

from either neighbour is not resolved on held-out lengths (for example, monitorability-aware minus naive at 10–14: +0.09 [-0.05, 0.22]).

6.4 What the agents learn: checking cued steps

Figure 3 shows how often each agent checks at a decision point whose current step is unchecked, split by whether that step's cue fired. With invariant features all three rewards produce cue-conditional checking. What distinguishes the monitorability-aware reward is *coverage* of cued steps, not avoidance of un-cued ones. It checks at 0.96 of cued decision points versus 0.71 for outcome-only, while checking un-cued ones at a similar rate (0.17 vs. 0.12). Outcome-only agents with absolute features barely condition on the cue at all (0.29 vs. 0.20). With absolute features, the monitorability-aware agent's cue coverage decays on held-out lengths (0.78 at 7–9). With invariant features it is largely preserved (0.89).

6.5 The residual gap: spending slack that long chains need

The invariant agents still lose ground on the longest chains (0.91 → 0.86 → 0.74). The cue-triggered heuristic does not (0.87 → 0.88 → 0.85). The budget-aware heuristic, tuned on training lengths, degrades like the learned agent (0.97 → 0.82 → 0.68), which points to how the budget is allocated rather than to the representation. The six units of slack beyond the chain length must cover every cued check plus one repair.

- **Un-cued checks.** On short chains, slack is left over for checking un-cued steps, which pays off. On long chains there are more cued steps to cover, and slack spent early is missing later.
- **Non-repair `REVISE`s.** The learned agents also spend slack on `REVISE`s that cannot be repairs (0.74 per episode at 10–14, each costing two units with its re-execution). This is a consequence of the memoryless observation (§3): after a SAFE check, undoing and redoing a step leads to the same observation as a genuine repair.

Both effects reduce fault detection at 10–14 to 0.75 for the learned agent and 0.60 for the budget-aware heuristic, against 0.86 for the cue-triggered one. The best allocation genuinely depends on how many steps remain, which the invariant encoder hides. §6.9 tests a memory bit and a remaining-length feature.

6.6 Robustness to environment shifts

Table 3 evaluates policies at lengths 7–9 under shifts they were not trained on.

Table 3: Success under shifts at held-out lengths 7–9, without retraining (mean over 10 seeds). p_f: fault probability (train: 0.8); c_V: verification cost (train: 1); slack: extra budget beyond the chain length (train: 6); clean/noisy cue: hit/false-positive rates 0.95/0.05 and 0.60/0.35 (train: 0.85/0.15).

Method	default	p_f=0.6	p_f=0.9	c_V=2	c_V=3	slack 4	slack 8	clean cue	noisy cue
Always THINK	0.20	0.41	0.10	0.20	0.19	0.20	0.21	0.21	0.20
Cue-triggered	0.88	0.91	0.86	0.75	0.49	0.73	0.88	0.96	0.62
Budget-aware	0.82	0.88	0.81	0.31	0.22	0.67	0.95	0.93	0.65
Outcome (inv)	0.62	0.72	0.56	0.51	0.22	0.42	0.41	0.75	0.41
Naive process (inv)	0.76	0.83	0.74	0.56	0.26	0.44	0.68	0.87	0.55
Monitorability (abs)	0.31	0.41	0.27	0.25	0.11	0.23	0.36	0.37	0.22
Monitorability (inv)	0.86	0.90	0.85	0.69	0.24	0.49	0.65	0.94	0.64

- **Fault rate.** The invariant monitorability-aware agent is robust to the fault rate (0.90 at p_f=0.6, 0.85 at p_f=0.9).
- **Cue quality.** It tracks the cue-triggered heuristic when cue quality changes (clean: 0.94 vs. 0.96; noisy: 0.64 vs. 0.62). This confirms that the policy relies on the cue rather than on a fixed checking schedule.
- **Verification cost.** It degrades when checks cost more than in training (0.24 at c_V=3), because it keeps checking at a rate the budget no longer supports.
- **Budget slack.** It is sensitive to the slack in *both* directions. With slack 4, success falls to 0.49 (cue-triggered: 0.73), although every slack value then stays inside the training range. This is the allocation problem of §6.5 again: a checking rate tuned to slack 6 runs out of budget. With slack 8, success falls to 0.65. Here the budget-aware heuristic improves (0.95), and the cue-triggered heuristic, which never spends extra slack, is unchanged (0.88). With slack 8 the slack feature also takes values never seen in training. The "invariant" encoder is invariant to length, not to budget.

6.7 Which reward terms matter

Table 4 removes one group of terms at a time from the monitorability-aware reward, with invariant observations and 10 seeds each. Because every variant is evaluated on the same episodes as the full reward, the differences below are paired.

- **Cue terms.** The cue terms (the bonus for informative checks of cued steps and the small penalty for checking un-cued ones) are the only component whose removal substantially hurts generalization. Without them, held-out success drops by 0.22 [0.13, 0.32] at lengths 7–9 and by 0.33 [0.19, 0.47] at 10–14 ($p = 0.004$), while training-length success barely changes (0.88 vs. 0.91). The mechanism is visible in the checking rates. Without the cue terms, agents check un-cued steps at about twice the rate (0.30 vs. 0.14 at 7–9). That costs little on short chains, but spends the slack that long chains need.
- **Cue bonus alone.** Conversely, the cue bonus alone, with every other process term removed, is indistinguishable from the full reward (0.85 at 7–9 and 0.77 at 10–14, vs. 0.86 and 0.74).
- **Other terms.** Removing the unchecked-gap term or the wasted-check penalty has no resolvable effect. Removing the safe/fault check bonuses, or all of the cue, gap and waste terms together, lowers held-out success moderately. Removing the per-action cost hurts at every length.
- **No `REVISE`.** Without `REVISE`, a caught fault cannot be repaired and success falls to the never-check level (0.20).

In short, most of what the monitorability-aware reward contributes in this environment comes from one term: paying for checks where a trusted cue says checking is valuable.

Table 4: Reward ablations (invariant observations; mean over 10 seeds, 95% t-interval). $P(V \mid c)$: probability of checking at a decision point whose current step is unchecked with cue c, at lengths 7–9.

Variant	Success 3–6	Success 7–9	Success 10–14	$P(V \mid c{=}1)$	$P(V \mid c{=}0)$
Full reward	0.91 [0.89, 0.92]	0.86 [0.83, 0.89]	0.74 [0.68, 0.80]	0.89	0.14
− cue bonus/penalty	0.88 [0.85, 0.90]	0.64 [0.56, 0.72]	0.40 [0.30, 0.51]	0.83	0.30
− unchecked-gap term	0.89 [0.87, 0.92]	0.84 [0.81, 0.87]	0.73 [0.61, 0.86]	0.88	0.13
− wasted-check penalty	0.89 [0.87, 0.92]	0.85 [0.81, 0.89]	0.68 [0.53, 0.82]	0.92	0.15
− safe/fault check bonuses	0.89 [0.88, 0.90]	0.78 [0.75, 0.82]	0.65 [0.56, 0.74]	0.92	0.15
− cue, gap and waste terms	0.89 [0.87, 0.90]	0.79 [0.72, 0.85]	0.66 [0.55, 0.76]	0.83	0.16
cue bonus only	0.88 [0.86, 0.90]	0.85 [0.82, 0.88]	0.77 [0.72, 0.83]	0.92	0.10
− per-action cost	0.85 [0.80, 0.89]	0.76 [0.69, 0.83]	0.61 [0.50, 0.71]	0.92	0.11
REVISE disabled	0.20 [0.19, 0.21]	0.19 [0.16, 0.21]	0.15 [0.12, 0.19]	0.61	0.11

6.8 Training stability and sensitivity

DQN agents in this environment exhibit severe policy churn [Schaul et al., 2022]. Figure 4 plots greedy success for each seed of the two monitorability-aware conditions. Policies repeatedly switch, within a few thousand episodes, between a solution that checks and repairs (≈0.9) and one that checks but does not repair (≈0.25–0.3; §6.2). After the first 5,000 episodes, the mean absolute change in training-length success between consecutive evaluations is 0.23 for the invariant monitorability-aware agent. That is larger than for outcome-only agents (0.10), which mostly sit in the never-check regime.

Because of this churn, last-checkpoint results (Appendix Table 8) are lower and noisier than selected-checkpoint results. For example, the invariant monitorability-aware agent scores 0.63 [0.51, 0.76] instead of 0.86 [0.83, 0.89] at lengths 7–9. With last checkpoints, the results split three ways:

- **Unchanged:** the encoder gap (+0.49 at 10–14, $p = 0.002$) and the advantage of the monitorability-aware reward over outcome-only (+0.30 at 7–9, $p = 0.004$).
- **Weakened:** naive process reward is no longer separable from outcome-only (+0.03 [-0.16, 0.22]).
- **Lost:** closeness to the cue-triggered heuristic. The last checkpoint is -0.25 [-0.37, -0.12] below it at lengths 7–9.

Selection uses only training-length validation episodes, so it cannot favour the encoder that generalizes better.

Sensitivity to the learner's hyperparameters. The main experiment uses the original study's DQN settings. To check that the encoder effect is not specific to them, we retrain the monitorability-aware agents with both encoders under three alternatives, five seeds each: learning rate $3{\cdot}10^{-4}$; hard target updates every 1,000 steps; and Polyak-averaged targets ($\tau = 0.005$) [Lillicrap et al., 2016] (Table 5). None of them removes the churn. In every setting the encoder gap reproduces: on lengths 10–14 the invariant encoder beats the absolute one by +0.64, +0.61 and +0.71 respectively, in 5/5 seeds each. With five seeds, the smallest attainable sign-flip p-value is 0.0625.

6.9 Follow-up interventions

The analyses above suggest three explanations, each with a direct test. We train each intervention with the monitorability-aware reward, invariant observations and 10 seeds, and compare it with the reference agent on paired episodes (Table 6).

Out-of-range inputs, not missing information, break generalization. Adding slack per remaining step, $\text{clip}(\text{slack}/(d-t), 0, 2)/2$, gives the agent exactly the information that §6.5 identified as missing. It also produces unseen observations at 51% of decision points at lengths 7–9 and 72% at 10–14, against 2% and 5% for the invariant encoder. We computed

Table 5: Sensitivity of the encoder comparison to DQN hyperparameters (monitorability-aware reward; mean over 5 seeds, 95% t-interval). Main-experiment values with the default settings are in Table 2.

Variant	Success 3–6	Success 7–9	Success 10–14	$P(V \mid c{=}1)$	$P(V \mid c{=}0)$
abs, lr $3{\cdot}10^{-4}$	0.72 [0.58, 0.87]	0.24 [0.05, 0.42]	0.03 [0.00, 0.08]	0.76	0.11
inv, lr $3{\cdot}10^{-4}$	0.88 [0.85, 0.91]	0.82 [0.75, 0.89]	0.67 [0.52, 0.82]	0.95	0.11
abs, hard sync 1000	0.80 [0.77, 0.83]	0.29 [0.16, 0.42]	0.02 [0.00, 0.04]	0.70	0.20
inv, hard sync 1000	0.90 [0.88, 0.91]	0.82 [0.74, 0.91]	0.63 [0.50, 0.76]	0.95	0.18
abs, Polyak τ=0.005	0.76 [0.73, 0.79]	0.26 [0.15, 0.37]	0.04 [0.00, 0.10]	0.66	0.18
inv, Polyak τ=0.005	0.90 [0.88, 0.93]	0.86 [0.84, 0.88]	0.75 [0.67, 0.83]	0.83	0.13

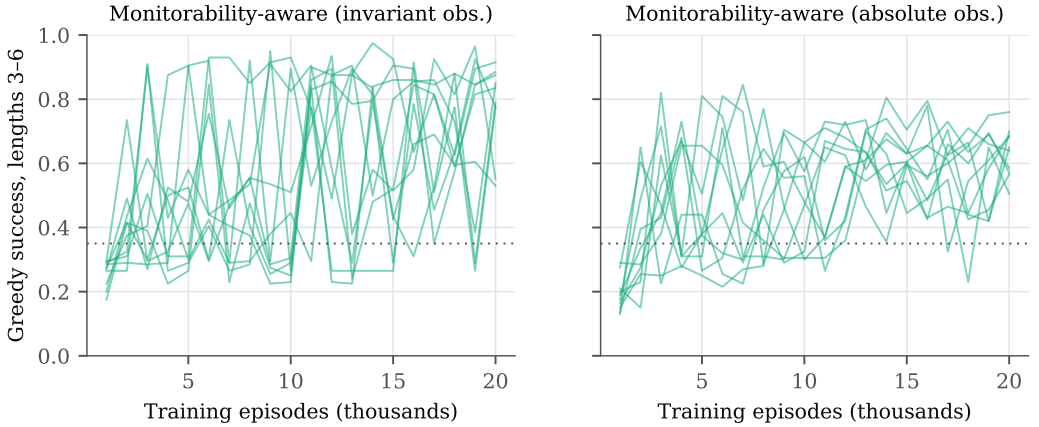


Figure 4: Greedy success on training lengths for each of the 10 seeds of the monitorability-aware agents. The dotted line marks success 0.35. Individual runs jump between solutions throughout training; for these agents the low regime is detect-without-repair, not never-check (§6.2).

this coverage, and committed it to the repository (commit `473bbe6`), before training the follow-up agents. On the out-of-range explanation it predicts worse generalization despite the extra information, and that is what happened. Held-out success falls to 0.46 at 7–9 and 0.28 at 10–14 (-0.46, $p = 0.002$). Fault detection stays high (0.81), but repairs are rarely completed (0.15).

A memory bit removes the wasted `REVISE`s. Adding one bit that records whether an UNSAFE check has been seen removes the aliasing between repairs and no-ops. Wasted `REVISE`s at 10–14 fall from 0.74 to 0.23 per episode. Training-length success rises by +0.03 (9/10 seeds, $p = 0.004$). Held-out success at selected checkpoints does not change resolvably ($p = 0.996$ at 10–14), because the agent spends the recovered slack on more un-cued checks (0.22 vs. 0.12). The memory bit also halves checkpoint collapse (10% vs. 21%).

Paying for repair removes the detect-without-repair collapse. Moving the 0.25 fault bonus from detection to the first correct re-execution of the located fault directly tests the explanation of §6.2. Collapsed late checkpoints drop from 21% to 1%. The mean absolute change between consecutive evaluations drops from 0.23 to 0.06. With the memory bit added as well, collapse disappears entirely (0%; change 0.03).

The benefit shows up in *reliability* rather than peak performance. At selected checkpoints the repair-paid reward is not better than the original (-0.07 at 7–9, $p = 0.080$), because selection already discards collapsed checkpoints. At the last checkpoint it is better: +0.12 [0.03, 0.21] at 7–9 ($p = 0.016$) and +0.12 at 10–14 ($p = 0.049$). The gap between selected and last checkpoints shrinks from 0.86 vs. 0.63 to 0.79 vs. 0.75. In practice this matters: with

Table 6: Follow-up interventions (mean over 10 seeds, 95% t-interval). "Last ckpt." evaluates the last checkpoint instead of the selected one; "collapsed" is the fraction of late checkpoints (after 5,000 episodes) with training-length success ≤ 0.35; "wasted `REVISE`" counts non-repair `REVISE`s per episode at lengths 10–14.

Variant	Success 3–6	Success 7–9	Success 10–14	Last ckpt. 7–9	Collapsed	Wasted REVISE
Monitorability (inv)	0.91 [0.89, 0.92]	0.86 [0.83, 0.89]	0.74 [0.68, 0.80]	0.63 [0.51, 0.76]	21%	0.74
+ slack-per-step feature	0.83 [0.80, 0.87]	0.46 [0.35, 0.57]	0.28 [0.20, 0.35]	0.37 [0.31, 0.43]	38%	0.64
+ fault-located memory bit	0.94 [0.92, 0.95]	0.88 [0.86, 0.91]	0.74 [0.65, 0.83]	0.81 [0.70, 0.91]	10%	0.23
fault bonus paid on repair	0.89 [0.88, 0.91]	0.79 [0.71, 0.87]	0.64 [0.52, 0.75]	0.75 [0.67, 0.83]	1%	0.51
memory bit + bonus on repair	0.93 [0.92, 0.95]	0.83 [0.76, 0.91]	0.66 [0.54, 0.77]	0.79 [0.72, 0.85]	0%	0.15

the process reward paying for the outcome its designer intended, the policy that training ends with is close to the best one it found.

7 Discussion

What process rewards buy here. The monitorability-aware reward does not unlock behaviour that outcome reward can never reach: with a suitable observation, outcome-only agents also find checking policies. What it buys is that checking is *retained*, and that cued steps are checked reliably. Outcome-only agents with invariant features stop checking in most late checkpoints; monitorability-aware agents never do. Almost all of this comes from one term that pays for checking where a trusted cue says checking is valuable (§6.7). For process supervision more broadly, the lesson is that rewarding a step-level behaviour helps exactly as much as the signal it is conditioned on is trustworthy: the agents tracked cue quality in both directions (§6.6).

Process rewards are themselves gameable. The original reward paid for *catching* a fault, not for fixing it, and its agents repeatedly drifted into policies that collect the detection bonus and then fail to repair (§6.2). This is a small, fully measurable instance of specification gaming of a process reward. It was invisible in success rates at selected checkpoints and surfaced only in the training dynamics. Paying for the repair instead removed it and brought the last checkpoint close to the selected one (§6.9). Designers of step-level rewards for reasoning models face the same question at larger scale: which intermediate events does the reward pay for, and are they the ones the designer actually cares about?

What process rewards do not buy. Process rewards did not fix length generalization. Under absolute position features, the best reward still fell below the never-check baseline on long chains. Every held-out observation was novel to the policy. Adding *more* information (slack per remaining step) made generalization worse when that information was out of range, as the coverage analysis predicted before the experiment was run. For process-supervised reasoning models, this suggests that "learned to verify on training problems" and "will verify appropriately on longer problems" are separate claims, to be evaluated separately. Positional or length features are a plausible way for the second claim to fail while the first holds.

Evaluation lessons. Two problems in the earlier version of this study pushed its conclusion toward a false negative: a representation confound, and last-checkpoint evaluation of churning policies. A third, a localization metric that credited unchecked faults, inflated diagnostic scores without changing the main comparison. None of the three was visible in the headline numbers. Small exact testbeds make such problems cheap to find. We recommend four practices for RL on reasoning-like tasks:

- report learning curves and per-seed variability;
- select checkpoints only on in-distribution validation;
- check how much of a held-out set is actually out of range for the policy's inputs;
- test whether out-of-distribution failures move when the representation changes but the reward does not.

Limitations. The environment is deliberately minimal. It has at most one fault, checks are exact, the cue is given rather than learned, and the agent is a small MLP rather than a language model. Process rewards use privileged step correctness, so they stand in for an idealised process reward model. We study one learner (Double DQN) with a small sensitivity sweep; policy-gradient or recurrent agents might churn less, or allocate their budget differently. With 10 seeds, differences of a few points (for example, between the naive and monitorability-aware rewards on long chains) are not resolved. Finally, no encoder we tried closes the gap to the cue-triggered heuristic on the longest chains. Doing so seems to require a representation of remaining length that stays in range as chains grow, which we leave open.

8 Conclusion

In a small budgeted-verification testbed, process rewards made checking behaviour learnable and kept it from collapsing, mainly by paying for checks where a trusted cue pointed. Whether that behaviour carried over to longer chains was decided mostly by the observation encoding, not by the reward. One part of the reward was gamed until it paid for repairs rather than detections. Correcting a positional confound turned an apparent negative result about monitorability-aware RL into a positive but qualified one. We hope the testbed is useful for probing how process supervision shapes *when* learned reasoners check their work.

Reproducibility. All code, configs, trained checkpoints, learning curves and per-seed evaluation results are in the accompanying repository (`https://github.com/raj200501/UltraRL`).

- `make report` and `make paper` rebuild every figure, table and number in this paper from the saved results.
- `make reproduce` retrains every agent from scratch, which takes about five hours on a 4-core CPU, and checks the checkpoints bitwise against the saved ones.
- `make equivalence` re-runs the comparison against the original environment.

References

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, 2021.

Cem Anil, Yuhuai Wu, Anders Andreassen, Aitor Lewkowycz, Vedant Misra, Vinay Ramasesh, Ambrose Slone, Guy Gur-Ari, Ethan Dyer, and Behnam Neyshabur. Exploring length generalization in large language models. In *Advances in Neural Information Processing Systems*, 2022.

Bowen Baker, Joost Huizinga, Leo Gao, Zehao Dou, Melody Y. Guan, Aleksander Madry, Wojciech Zaremba, Jakub Pachocki, and David Farhi. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. *arXiv preprint arXiv:2503.11926*, 2025.

Yanda Chen, Joe Benton, Ansh Radhakrishnan, Jonathan Uesato, Carson Denison, John Schulman, Arushi Somani, Peter Hase, Misha Wagner, Fabien Roger, Vlad Mikulik, Samuel R. Bowman, Jan Leike, Jared Kaplan, and Ethan Perez. Reasoning models don't always say what they think. *arXiv preprint arXiv:2505.05410*, 2025.

Karl Cobbe, Oleg Klimov, Chris Hesse, Taehoon Kim, and John Schulman. Quantifying generalization in reinforcement learning. In *International Conference on Machine Learning*, 2019.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Grégoire Delétang, Anian Ruoss, Jordi Grau-Moya, Tim Genewein, Li Kevin Wenliang, Elliot Catt, Chris Cundy, Marcus Hutter, Shane Legg, Joel Veness, and Pedro A. Ortega. Neural networks and the Chomsky hierarchy. In *International Conference on Learning Representations*, 2023.

Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

Nicholas Hay, Stuart Russell, David Tolpin, and Solomon Eyal Shimony. Selecting computations: Theory and applications. In *Conference on Uncertainty in Artificial Intelligence*, 2012.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *AAAI Conference on Artificial Intelligence*, 2018.

Robert Kirk, Amy Zhang, Edward Grefenstette, and Tim Rocktäschel. A survey of zero-shot generalisation in deep reinforcement learning. *Journal of Artificial Intelligence Research*, 76:201–264, 2023.

Tomek Korbak, Mikita Balesni, et al. Chain of thought monitorability: A new and fragile opportunity for AI safety. *arXiv preprint arXiv:2507.11473*, 2025.

Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, et al. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*, 2023.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations*, 2024.

Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. In *International Conference on Learning Representations*, 2016.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning*, 1999.

OpenAI. Learning to reason with LLMs. https://openai.com/index/learning-to-reason-with-llms/, 2024.

OpenAI. Evaluating chain-of-thought monitorability. https://openai.com/index/evaluating-chain-of-thought-monitorability/, 2025.

Stuart Russell and Eric Wefald. Principles of metareasoning. *Artificial Intelligence*, 49(1–3): 361–395, 1991.

Tom Schaul, André Barreto, John Quan, and Georg Ostrovski. The phenomenon of policy churn. In *Advances in Neural Information Processing Systems*, 2022.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, 2023.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022.

Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *AAAI Conference on Artificial Intelligence*, 2016.

Tian Xie, Zitian Gao, Qingnan Ren, Haoming Luo, Yuqian Hong, Bryan Dai, Joey Zhou, Kai Qiu, Zhirong Wu, and Chong Luo. Logic-RL: Unleashing LLM reasoning with rule-based reinforcement learning. *arXiv preprint arXiv:2502.14768*, 2025.

A Environment Details

Transitions. Given the current true value v, a transition is drawn uniformly from the available operations: `add` k with $k \in \{1, \ldots, 4\}$; `sub` k with $k \in \{1, \ldots, \min(4, v-1)\}$ if $v > 2$; and `mul` k with $k \in \{2, 3\}$ if $|v| \leq 16$. The corrupted version of `add`/`sub` k uses $k{+}1$, and that of `mul` k is `add` k. If the corrupted operation happens to give the same value, it is replaced by `add` $k{+}2$. A corrupted step therefore always changes the value (unit-tested).

Fault prior. Conditional on a fault, its position $h \in \{0, \ldots, d-1\}$ has weight $1 + 5\,(1 - |h - c| / \max(1, c))$ with $c = (d-1)/2$, i.e. it peaks in the middle of the chain.

Cue selectivity metric. At every decision point where the current step $t > 0$ has not been checked and `VERIFY` is available, we record the cue c_t and whether the agent chose `VERIFY`. $P(\texttt{VERIFY} \mid c)$ pools these counts over all evaluation episodes of a seed.

Validation against the original implementation. The environment was rewritten for this version. `make equivalence` (`scripts/check_legacy_equivalence.py`) extracts the original implementation from git history and plays identical random action sequences in both. Over 86,112 steps from 18,000 episodes, covering all reward schemes and both budget-penalty settings, the two agree exactly on transitions, masks, rewards, termination and absolute observations. The only difference is the observation returned after the terminal `COMMIT`, which no learner uses.

B Hyperparameters and Compute

Table 7: Default environment and learner settings.

Setting	Value
Training / held-out lengths	3–6 / 7–9 and 10–14
Fault probability; cue hit / false-positive rate	0.8; 0.85 / 0.15
Budget; action costs (`THINK`, `VERIFY`, `REVISE`)	$d + 6$; $(1, 1, 1)$
Rewards: success / failure / per-unit cost	$+1$ / -1.2 / -0.04
Monitorability-aware coefficients (Eq. 1)	$0.02, 0.25, 0.18, 0.02, 0.06, 0.30$
Network	MLP, 2 hidden layers of 64 ReLU units
Optimiser; learning rate; gradient clip	Adam; 10^{-3}; 1.0 (global norm)
Discount; loss	0.99; Huber
Replay size; batch size; warm-up	25,000; 64; 500 steps
Target network	hard copy every 200 environment steps
Exploration	ϵ: $0.5 \to 0.02$ linearly over 85% of training
Training episodes per run	20,000
Seeds	10 (main, ablations), 5 (sensitivity)
Validation for selection	200 episodes of lengths 3–6 every 1,000 episodes
Evaluation	500 episodes per seed and setting

A training run takes about four minutes on one CPU core. The main experiment (60 runs), the ablations (80 runs; the full-reward condition is shared with the main experiment), the sensitivity study (30 runs) and the follow-ups (40 runs) together take about five hours on

a 4-core machine. No GPU is used. Training is deterministic on CPU given a seed; `make reproduce` retrains everything into a fresh directory and checks every checkpoint bitwise against the saved ones.

C Last-Checkpoint Results

Table 8: As Table 2, but evaluating the last checkpoint of each run instead of the one selected on training-length validation. Heuristics are repeated for reference.

Method	Success 3–6	Success 7–9	Success 10–14	Detect 7–9	Cost-adj. 7–9
Random	0.00 [0.00, 0.01]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.02 [0.01, 0.02]	-0.09 [-0.09, -0.08]
Always THINK	0.20 [0.19, 0.21]	0.20 [0.19, 0.22]	0.20 [0.18, 0.21]	0.00 [0.00, 0.00]	-0.20 [-0.21, -0.18]
Verify every step	0.80 [0.79, 0.81]	0.06 [0.06, 0.07]	0.00 [0.00, 0.00]	0.93 [0.91, 0.94]	-0.63 [-0.64, -0.63]
Cue-triggered	0.87 [0.86, 0.89]	0.88 [0.87, 0.90]	0.85 [0.84, 0.86]	0.85 [0.84, 0.87]	0.33 [0.31, 0.34]
Budget-aware	0.97 [0.96, 0.97]	0.82 [0.81, 0.84]	0.68 [0.66, 0.70]	0.78 [0.76, 0.80]	0.19 [0.17, 0.20]
Outcome (inv)	0.40 [0.27, 0.52]	0.33 [0.21, 0.46]	0.30 [0.19, 0.41]	0.25 [0.04, 0.45]	-0.31 [-0.43, -0.18]
Naive process (inv)	0.41 [0.25, 0.58]	0.36 [0.21, 0.51]	0.31 [0.20, 0.43]	0.58 [0.42, 0.75]	-0.30 [-0.45, -0.14]
Monitorability (inv)	0.77 [0.68, 0.86]	0.63 [0.51, 0.76]	0.52 [0.41, 0.64]	0.81 [0.74, 0.89]	-0.02 [-0.15, 0.10]
Outcome (abs)	0.24 [0.22, 0.26]	0.12 [0.05, 0.19]	0.08 [0.02, 0.14]	0.06 [0.00, 0.16]	-0.20 [-0.34, -0.06]
Naive process (abs)	0.23 [0.17, 0.30]	0.12 [0.05, 0.20]	0.05 [0.00, 0.11]	0.19 [0.02, 0.35]	-0.28 [-0.41, -0.15]
Monitorability (abs)	0.63 [0.59, 0.68]	0.24 [0.15, 0.34]	0.03 [0.00, 0.07]	0.51 [0.36, 0.65]	-0.34 [-0.43, -0.24]

D Behavioural Diagnostics

Table 9: Behaviour at held-out lengths 7–9 (mean over 10 seeds, selected checkpoints). Non-repair `REVISE`s do not follow an UNSAFE check and cannot repair anything; "success | detect" is the fraction of episodes with a caught fault that end in success; "early commit" is the fraction of episodes committed before the chain is finished although the budget could still finish it.

Method	Checks/ep.	Non-repair REVISE/ep.	Detect	Success | detect	Early commit
Cue-triggered	1.76	0.00	0.85	1.00	0.00
Budget-aware	3.52	0.00	0.78	1.00	0.00
Outcome (inv)	2.23	0.83	0.61	0.84	0.00
Naive process (inv)	3.38	0.42	0.76	0.91	0.00
Monitorability (inv)	2.70	0.76	0.85	0.97	0.00
Outcome (abs)	1.30	1.04	0.09	0.58	0.33
Naive process (abs)	2.15	0.78	0.30	0.22	0.30
Monitorability (abs)	2.01	1.15	0.60	0.35	0.18

Table 10: As Table 9, at lengths 10–14.

Method	Checks/ep.	Non-repair REVISE/ep.	Detect	Success | detect	Early commit
Cue-triggered	2.37	0.00	0.86	0.95	0.00
Budget-aware	3.74	0.00	0.60	1.00	0.00
Outcome (inv)	2.51	0.85	0.53	0.79	0.00
Naive process (inv)	3.69	0.42	0.67	0.85	0.00
Monitorability (inv)	3.19	0.74	0.75	0.91	0.00
Outcome (abs)	0.39	0.79	0.02	0.42	0.74
Naive process (abs)	0.47	0.22	0.07	0.05	0.81
Monitorability (abs)	1.08	1.06	0.15	0.01	0.70

E Changes from the Earlier Version

The first public version of this repository reported that monitorability-aware RL reached 0.60 success on training lengths but only 0.11 on lengths 7–9, and concluded that the learned checking behaviour was brittle under length shift. This version changes the following.

1. **Representation control.** The original observation contained absolute position and depth features. We add a length-invariant encoder and compare the two directly (§6.3). The encoder, not the reward, explains most of the length-generalization failure.
2. **Training length and checkpoint selection.** Runs were trained for 5,000 episodes, while learning curves were still rising, and evaluated at the last checkpoint. We train for 20,000 episodes, select checkpoints on training-length validation only, and report last-checkpoint results as well (Appendix C).
3. **Seed overlap.** Training episode seeds were $\texttt{seed} \cdot 10^5 + i$ and evaluation seeds $\texttt{seed} \cdot 10^6 + i$, which coincide for seed 0: 299 of seed 0's 300 training-length evaluation episodes were exact copies of training episodes. Because policies observe only depth, position, budget, cue and check results, an episode is determined (for a deterministic policy) by its depth, fault position and cue vector, and on training lengths every such configuration recurs many times in training anyway. The overlap therefore could not bias the reported numbers; we fixed it for hygiene by using independent random-number streams, and tests check that evaluation draws from its own stream.
4. **Localization metric.** The original "failed-episode localization score" gave full credit when the fault was on the last step and the agent committed without checking anything, and partial credit for any commit at or after the fault step; as a result the never-check policy scored 0.07. It inflated every method's diagnostic score without changing the main comparison. We replace it with fault detection, the probability that a check caught the fault, and add repair completion.
5. **Stronger baselines and statistics.** We add a budget-aware heuristic tuned on training lengths, use 10 seeds instead of 5, and report t-intervals, paired differences and permutation tests instead of standard errors.
6. **Heuristic definitions.** The original heuristics committed as soon as the last step was executed, so they never checked the final step, and the cue-triggered heuristic re-checked a step after repairing it. Ours check the final step and never re-check. On the same evaluation episodes this raises cue-triggered success at lengths 3–6 from 0.82 to 0.87 (almost entirely from the final-step check) and verify-every-step success from 0.57 to 0.80. Learned agents are unaffected.